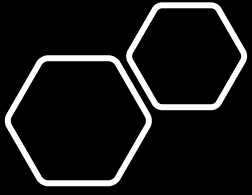


Interaction potentials: from first principles to machine learning

LJ and Morse fitted to DFT data (Xe, H), machine-learned potentials, coarse-graining

Overview

1. Lennard-Jones: which physical interaction does it describe?
2. Interatomic potentials from first principles: DFT
3. Case study #1: LJ for xenon from DFT data
4. Case study #2: hydrogen, LJ fails and Morse works
5. Beyond fitted formulas: machine-learned potentials (PaiNN, MACE)
6. Second level: coarse-graining with ML (prior + neural network)



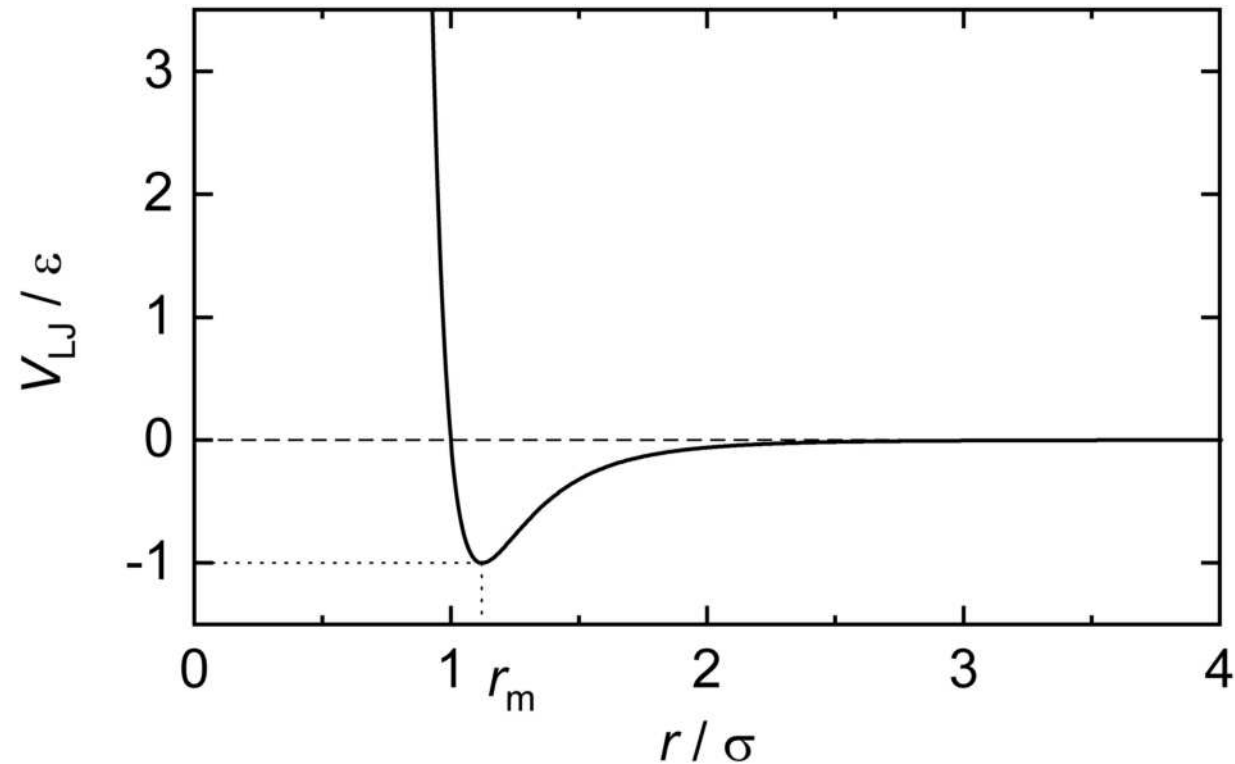
Lennard-Jones Potential

- Weakly attractive in a small region
- Strongly repulsive
- Quickly goes to zero
- **Which kind of physical interaction looks like this?**

$$V(r) = 4\epsilon \left(\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right)$$

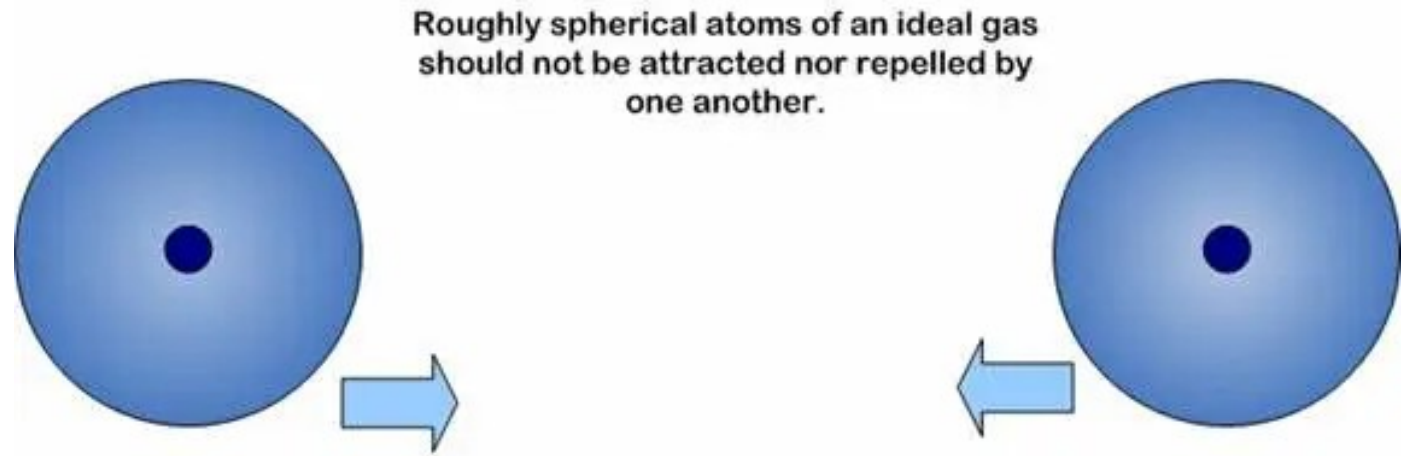
$$r_{min} = 2^{1/6} \sigma$$

$$V = -\epsilon$$



Lennard-Jones Potential

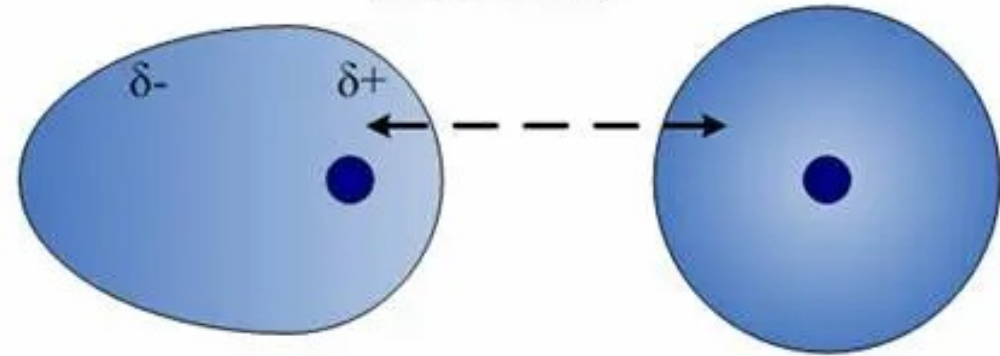
- Weakly attractive in a small region
- Strongly repulsive
- Quickly goes to zero
- **Induced-dipole with induced-dipole**



Lennard-Jones Potential

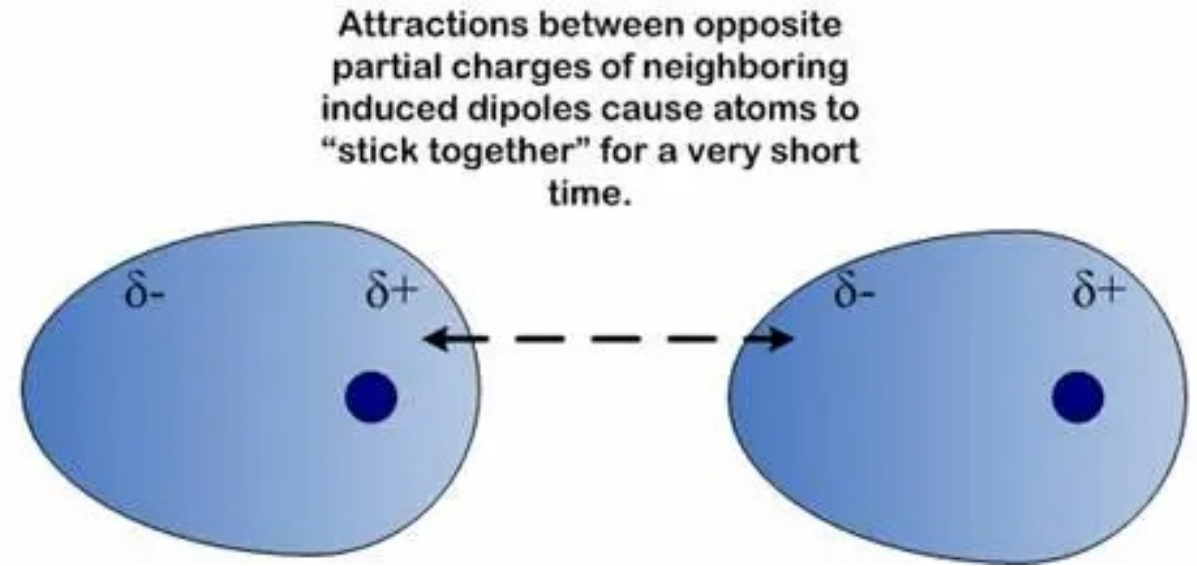
- Weakly attractive in a small region
- Strongly repulsive
- Quickly goes to zero
- **Induced-dipole with induced-dipole**

A real gas atom can have an instantaneous dipole. Partial charges on one atom cause a neighboring atom to distort due to the electrostatic attractions/repulsions of their electron clouds.



Lennard-Jones Potential

- Weakly attractive in a small region
- Strongly repulsive
- Quickly goes to zero
- **Induced-dipole with induced-dipole**



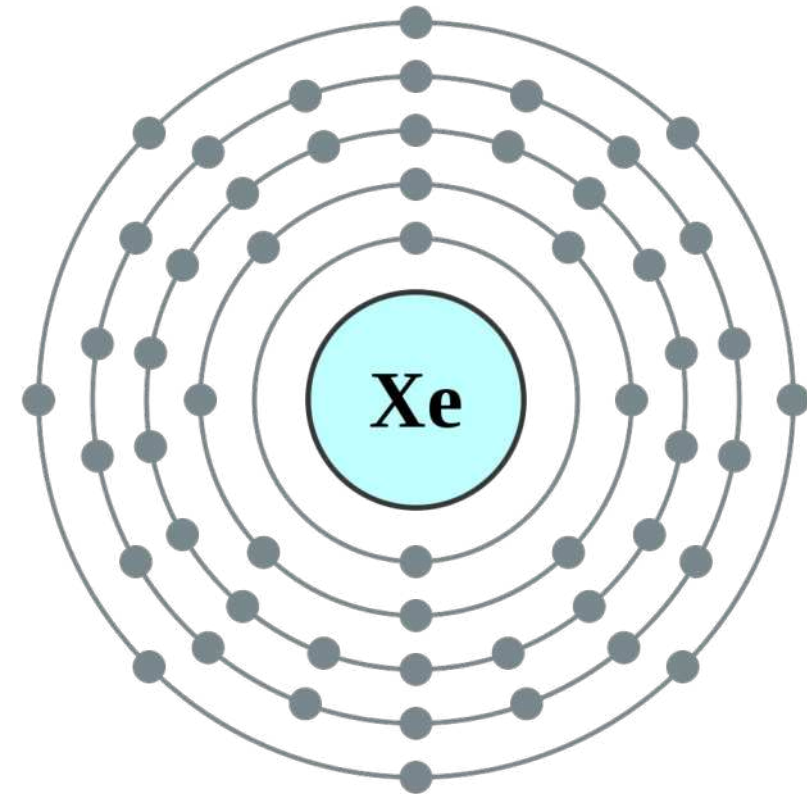
Which kind of atoms interact like this?

Case study #1: LJ Potential for Xe atoms

- To study a system of Xe atoms, we need to know the two Lennard-Jones parameters ϵ and σ
- **How can we determine them?**
 - Try to fit experiments
 - From first principles
- What are the advantages of the second approach?
 - More consistent
 - Much faster than an experiment
 - Much cheaper than an experiment
 - Experiments may not be available at all

54: Xenon

2,8,18,18,8



Interatomic potential from first principles

Many-body Hamiltonian

$$H = \underbrace{-\frac{1}{2} \sum_{i=1}^{N_e} \nabla_i^2}_{\text{kinetic}} \underbrace{-\frac{1}{2} \sum_{I=1}^{N_a} \nabla_I^2}_{\text{kinetic}} \underbrace{-\frac{1}{2} \sum_{i=1}^{N_e} \sum_{I=1}^{N_a} \frac{Z_I}{|\vec{r}_i - \vec{R}_I|}}_{\text{electron-ion}} + \underbrace{\frac{1}{2} \sum_{i=1}^{N_e} \sum_{j=1}^{N_e} \frac{1}{|\vec{r}_i - \vec{r}_j|}}_{\text{electron-electron}} + \underbrace{\frac{1}{2} \sum_{I=1}^{N_a} \sum_{J=1}^{N_a} \frac{1}{|\vec{R}_I - \vec{R}_J|}}_{\text{ion-ion}}$$

H_e H_I

The *electronic eigenenergies* depend parametrically on the ionic positions
This is what the interaction potential does

Born-Oppenheimer approximation:
 Decouple electronic and ionic degrees of freedom

$$H_e = -\frac{1}{2} \sum_{i=1}^{N_e} \nabla_i^2 - \frac{1}{2} \sum_{i=1}^{N_e} \sum_{I=1}^{N_a} \frac{Z_I}{|\vec{r}_i - \vec{R}_I|} + \frac{1}{2} \sum_{i=1}^{N_e} \sum_{j=1}^{N_e} \frac{1}{|\vec{r}_i - \vec{r}_j|}$$

$$H_e \psi(\vec{r}_1, \dots, \vec{r}_{N_e}) = E_e(\vec{R}_1, \dots, \vec{R}_{N_a}) \psi(\vec{r}_1, \dots, \vec{r}_{N_e})$$

Solving this equation for more than a **few** electrons is practically impossible

Density Functional Theory

How to solve the Schrödinger's equation for many electrons

Density Functional Theory allows us to compute the **ground-state electronic energy** and charge density of a N electron system, which N up to tens of thousands of electrons.

In a nutshell, DFT is based on the construction of an auxiliary problem of *noninteracting* electrons in a fictitious external potential (v_{scf}) such as to reproduce exactly the same ground-state charge density as the interacting problem. Since in the auxiliary problem electrons are *noninteracting*, the Schrödinger's equation becomes that of a *single* electron.

$$v_{ext}(\vec{r}) = \sum_I \int \frac{Z_I}{|\vec{r} - \vec{R}_I|} d^3r'$$
$$v_H(\vec{r}) = \frac{1}{2} \int \frac{n(\vec{r}')}{|\vec{r} - \vec{r}'|} d^3r'$$

$$\left\{ \begin{array}{l} v_{scf}(\vec{r}) = v_{ext}(\vec{r}) + v_H(\vec{r}) + v_{XC}(\vec{r}) \\ [-\frac{1}{2}\nabla^2 + v_{scf}(\vec{r})]\psi_i(\vec{r}) = \varepsilon_i\psi_i(\vec{r}) \\ n(\vec{r}) = \sum_{i=1}^{N/2} |\psi_i(\vec{r})|^2 \end{array} \right.$$

Density Functional Theory

How to solve the Schrödinger's equation for many electrons

Nowadays, DFT calculations for molecules and solids are widespread, and many *ab initio* codes can be used, such as:

- Quantum ESPRESSO (open-source)
 - Abinit (open source)
 - CASTEP (recently became free for academia)
 - VASP (licensed software)
-
- Without going into the details, in the following we will use VASP to compute the **total electronic energy of a pair of Xe atoms**, as a function of their distance r , which will give us the $E(r)$

Workflow: LJ Potential for two Xe atoms

1. Generate a set of atomic positions for two Xe atoms
2. Compute the total electronic energy for each configuration
3. Collect the data into a file with two columns: $[r_i, V(r_i)]$
4. Import the data in python and fit

Workflow: LJ Potential for two Xe atoms

1. Generate a set of atomic positions for two Xe atoms
 - What is a sensible **range** of distances?

Atomic properties	
Oxidation states	0 , +2, +4, +6, +8 (rarely more than 0; a weakly acidic oxide)
Electronegativity	Pauling scale: 2.6
Ionization energies	1st: 1170.4 kJ/mol 2nd: 2046.4 kJ/mol 3rd: 3099.4 kJ/mol
Covalent radius	140±9 pm
Van der Waals radius	216 pm

Workflow: LJ Potential for two Xe atoms

1. Generate a set of atomic positions for two Xe atoms
 - What is a sensible **range** of distances?

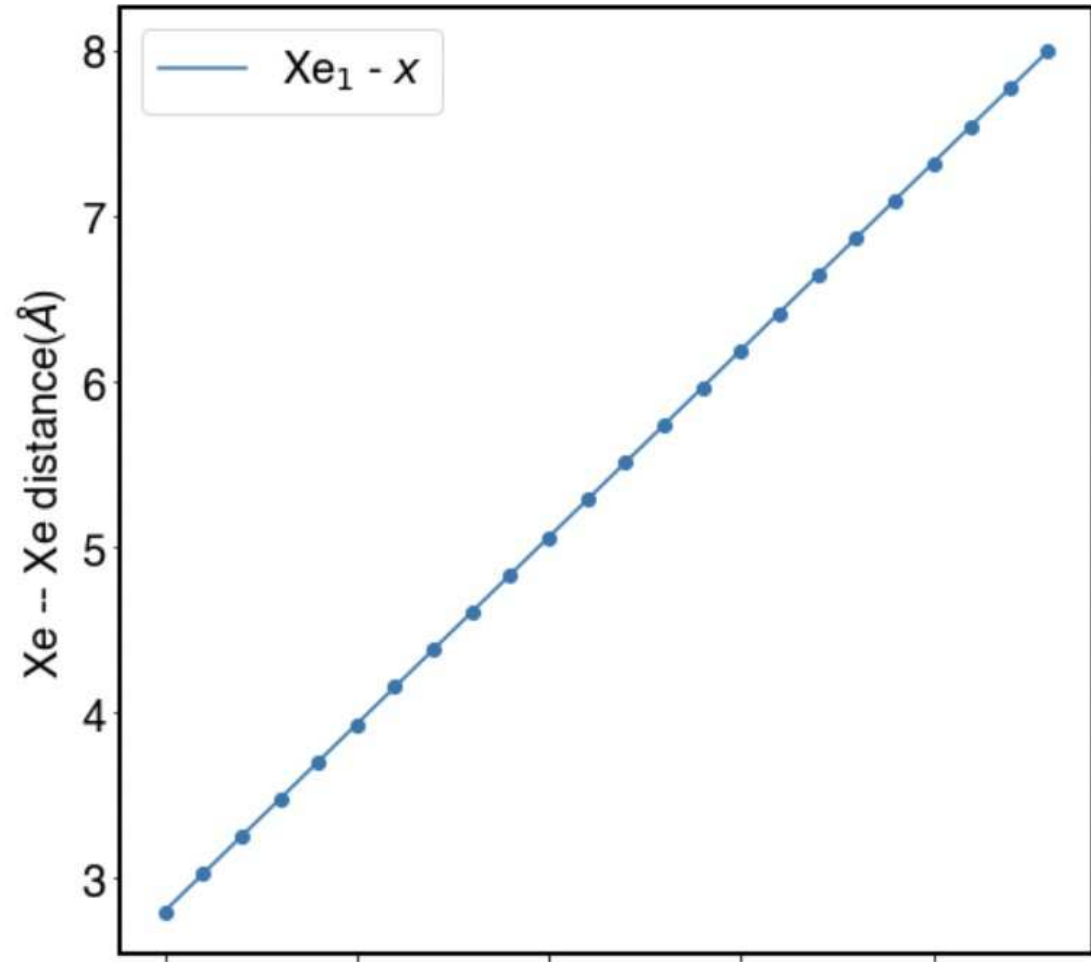
```
import numpy as np
filesave = []
N_points = 24
## Define lattice parameters (cubic box)
lattpar_matrix = np.asarray([[1.000, 0.000, 0.000],
                             [0.000, 1.000, 0.000],
                             [0.000, 0.000, 1.000]])
## 6 Å side cubic lattice
lattpar_matrix = 16*lattpar_matrix
```

```
## Xe-Xe distance: from 2.80 to 8.00 Å
## N_points, linearly-spaced points from 2.80 to 8 Å
lin_d_range = np.linspace(2.80, 8.00, N_points)

## First Xe atom can be in the origin
Xe_0 = np.tile(np.asarray([0.00, 0.00, 0.00]), (N_points, 1))
## The second Xe atom is shifted
Xe_1 = Xe_0.copy()
Xe_1[:,0] = Xe_0[:,0] + lin_d_range
```

Workflow: LJ Potential for two Xe atoms

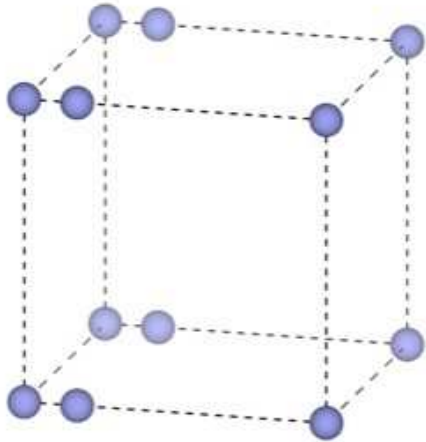
1. Generate a set of atomic positions for two Xe atoms
- What is a sensible **range** of distances?



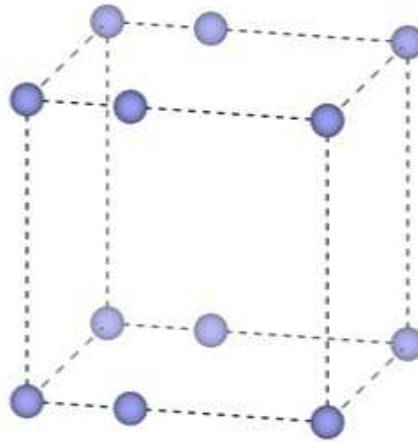
Let's visualize the atoms in the structures we just created

Why are there atoms in all corners of the cube?

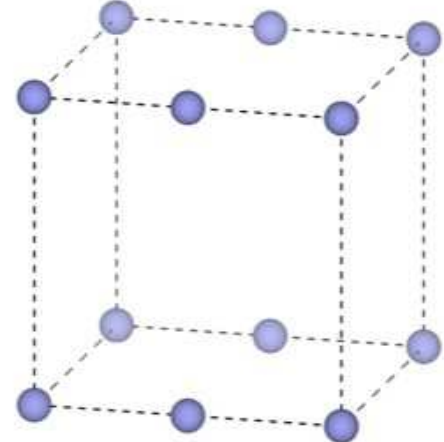
Step 0



Step 12



Step 23



DFT
output

FORCE on cell =-STRESS in cart. coord. units (eV):						
Direction	XX	YY	ZZ	XY	YZ	ZX
Alpha Z	2.33946	2.33946	2.33946			
Ewald	61.41547	-115.18357	-115.18357	-0.00000	-0.00000	0.00000
Hartree	292.71740	126.85671	126.85671	-0.00000	-0.00000	-0.00000
E(xc)	-67.98583	-67.98608	-67.98608	0.00000	0.00000	0.00000
Local	-316.59576	25.86374	25.86374	0.00000	0.00000	-0.00000
n-local	-225.98579	-225.98648	-225.98648	-0.00000	0.00000	-0.00000
augment	61.13282	61.13311	61.13311	0.00000	0.00000	0.00000
Kinetic	192.95976	192.96195	192.96195	-0.00000	-0.00000	0.00000
Fock	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
Total	-0.00248	-0.00116	-0.00116	0.00000	0.00000	-0.00000
in kB	-0.00097	-0.00045	-0.00045	0.00000	0.00000	-0.00000
external pressure =		-0.00 kB	Pullay stress =		0.00 kB	

External pressure

VOLUME and BASIS-vectors are now :

energy-cutoff :	300.00				
volume of cell :	4096.00				
direct lattice vectors			reciprocal lattice vectors		
16.000000000	0.000000000	0.000000000	0.062500000	0.000000000	0.000000000
0.000000000	16.000000000	0.000000000	0.000000000	0.062500000	0.000000000
0.000000000	0.000000000	16.000000000	0.000000000	0.000000000	0.062500000
length of vectors					
16.000000000	16.000000000	16.000000000	0.062500000	0.062500000	0.062500000

FORCES acting on ions			ewald-force			non-local-force			convergence-correction		
electron-ion (+dipol)											
0.515E-14	0.494E-14	0.284E-14	-.219E-15	-.297E-17	-.111E-16	0.000E+00	-.496E-23	-.662E-23	0.108E-13	0.754E-14	0.488E-14
0.204E-12	0.477E-14	0.123E-11	0.199E-15	0.121E-16	0.305E-17	0.000E+00	-.496E-23	-.662E-23	0.391E-14	0.139E-13	-.858E-13
0.210E-12	0.970E-14	0.123E-11	-.199E-16	0.911E-17	-.801E-17	0.000E+00	-.993E-23	-.132E-22	0.148E-13	0.214E-13	-.809E-13

POSITION	TOTAL-FORCE (eV/Angst)					
0.00000	0.00000	0.00000	-0.000000	0.000000	0.000000	
8.00000	0.00000	0.00000	0.000000	-0.000000	-0.000000	
total drift						

Forces between the
atoms

Can you guess to which of the atomic configurations we generated this output corresponds?

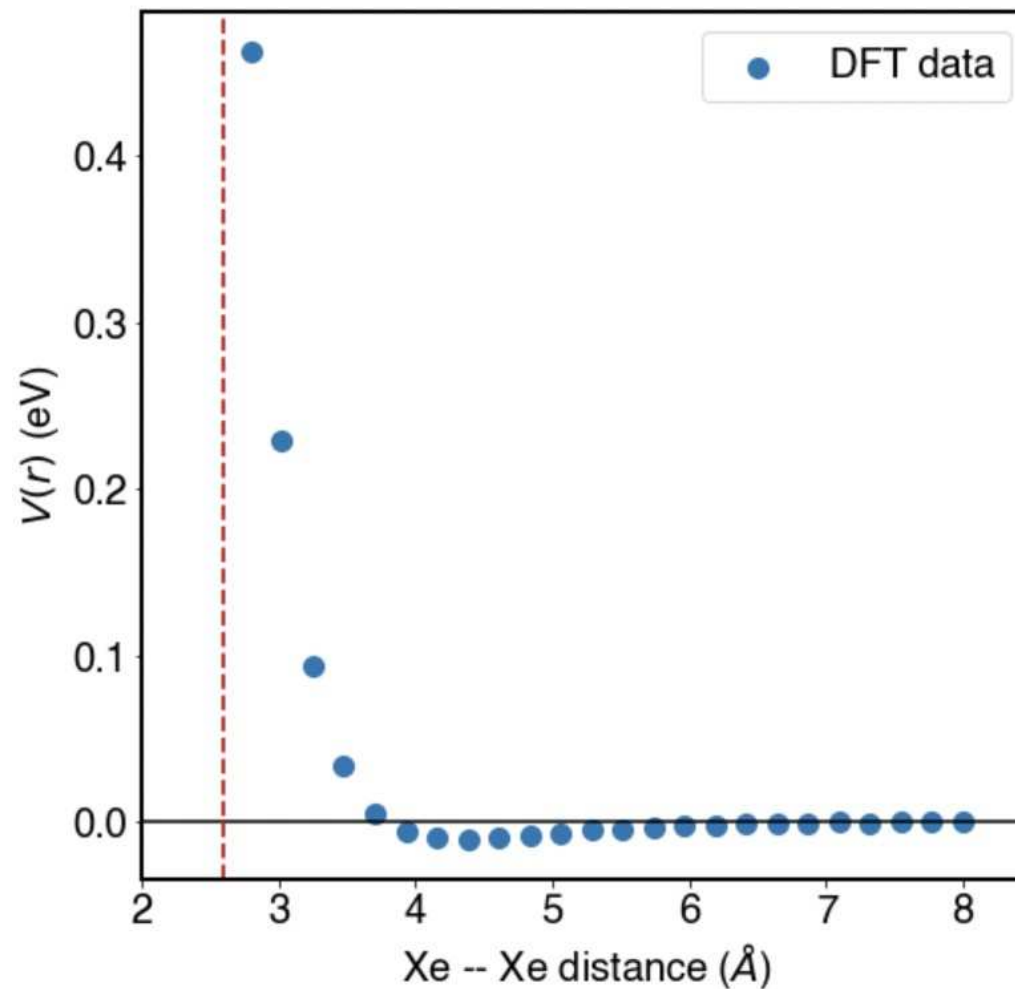
FREE ENERGIE OF THE ION-ELECTRON SYSTEM (eV)			
free energy	TOTEN =	-0.02427826	eV
energy without entropy=	-0.02427826	energy(sigma->0) =	-0.02427826

Total energy of the
system

Collected DFT data

Is this what we expected?

```
## I saved into a file "TOTEN" (total energy)
## together with the Xe-Xe distance
## Load the data
LJ_data = np.loadtxt("r_data.dat")
## Rescale: set  $V(+\infty) = 0$  and set the potential per atom
LJ_data[:,2] = (LJ_data[:,2] - LJ_data[-1,2])/2
```



Define fitting function

```
## Now let's do a fit!
from scipy.optimize import curve_fit
## Define the LJ function
def lennard_jones_1(x, epsilon, sigma):
    return 4*epsilon*((sigma/(x))**12-(sigma/(x))**6)

## Decide: which range of data points shall we use for the fit?
mask_fit = (LJ_data[:,0] > 3.0) & (LJ_data[:,0] < 8.0)
popt_1, pconv = curve_fit(lennard_jones_1,
                          LJ_data[mask_fit,0],
                          LJ_data[mask_fit,2],
                          p0=[0.02, 4.5],
                          bounds=[[0,2.0], [2, 10.0]],
                          maxfev=5000, ftol=1e-10)
```

Fit the data points and plot

- Does the fit reproduce well our data everywhere?
- Should we be worried?
- Can we do better?

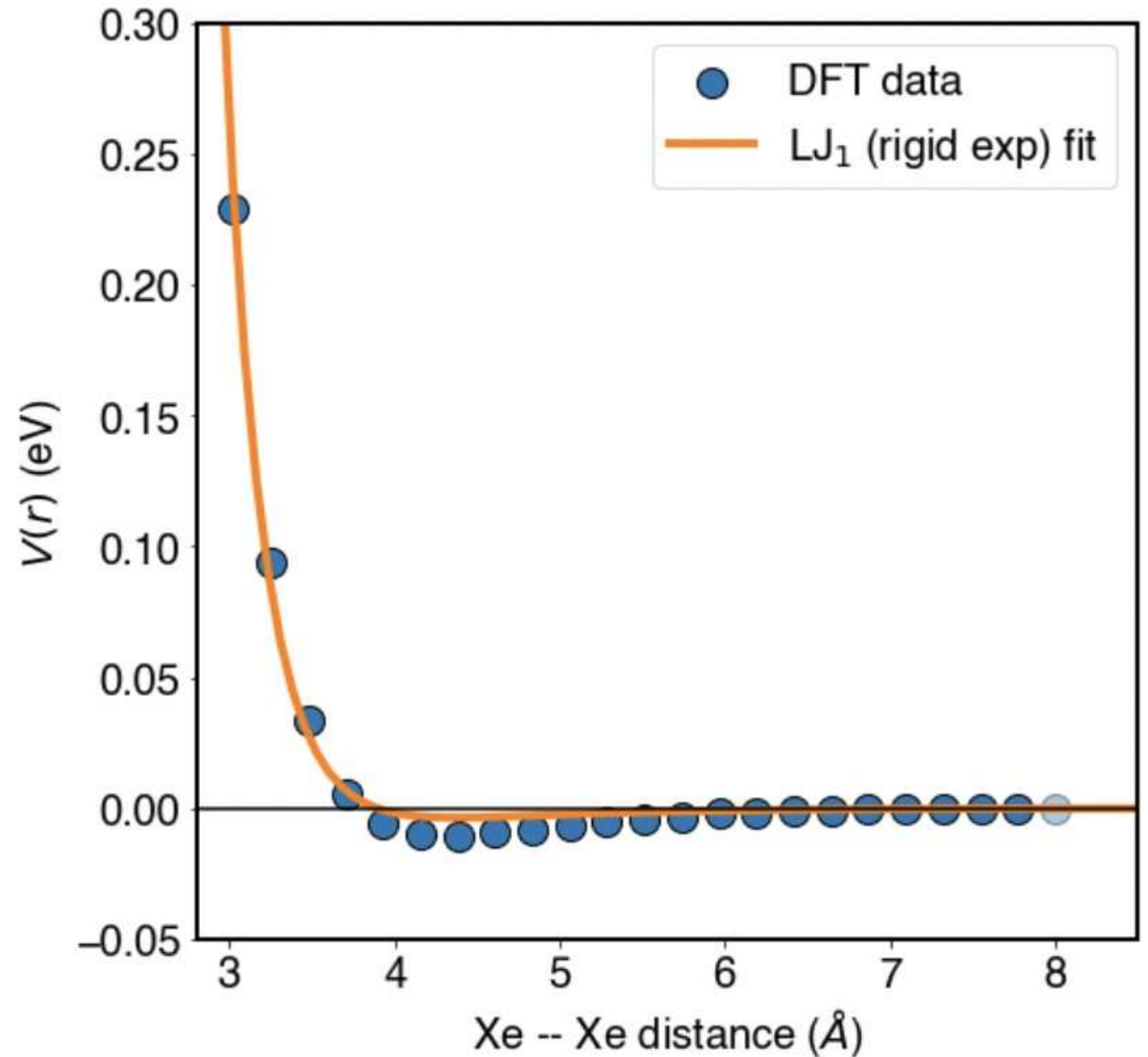
Fit results

ε (eV/atom): 0.0037

σ (Å): 3.8927

repulsive exponent: 12

binding distance: (Å): 4.36



Define fitting function

Exclude some short-range data points

```
## Now let's do a fit!
from scipy.optimize import curve_fit
## Define the LJ function
def lennard_jones_1(x, epsilon, sigma):
    return 4*epsilon*((sigma/(x))**12-(sigma/(x))**6)

## Decide: which range of data points shall we use for the fit?
mask_fit = (LJ_data[:,0] > 3.6) & (LJ_data[:,0] < 8.0)
popt_1, pconv = curve_fit(lennard_jones_1,
                          LJ_data[mask_fit,0],
                          LJ_data[mask_fit,2],
                          p0=[0.02, 4.5],
                          bounds=[[0,2.0], [2, 10.0]],
                          maxfev=5000, ftol=1e-10)
```

Fit the data points and plot

- Does the fit reproduce well our data everywhere?
- Should we be worried?
- Is this fit better?
- Can we do better?

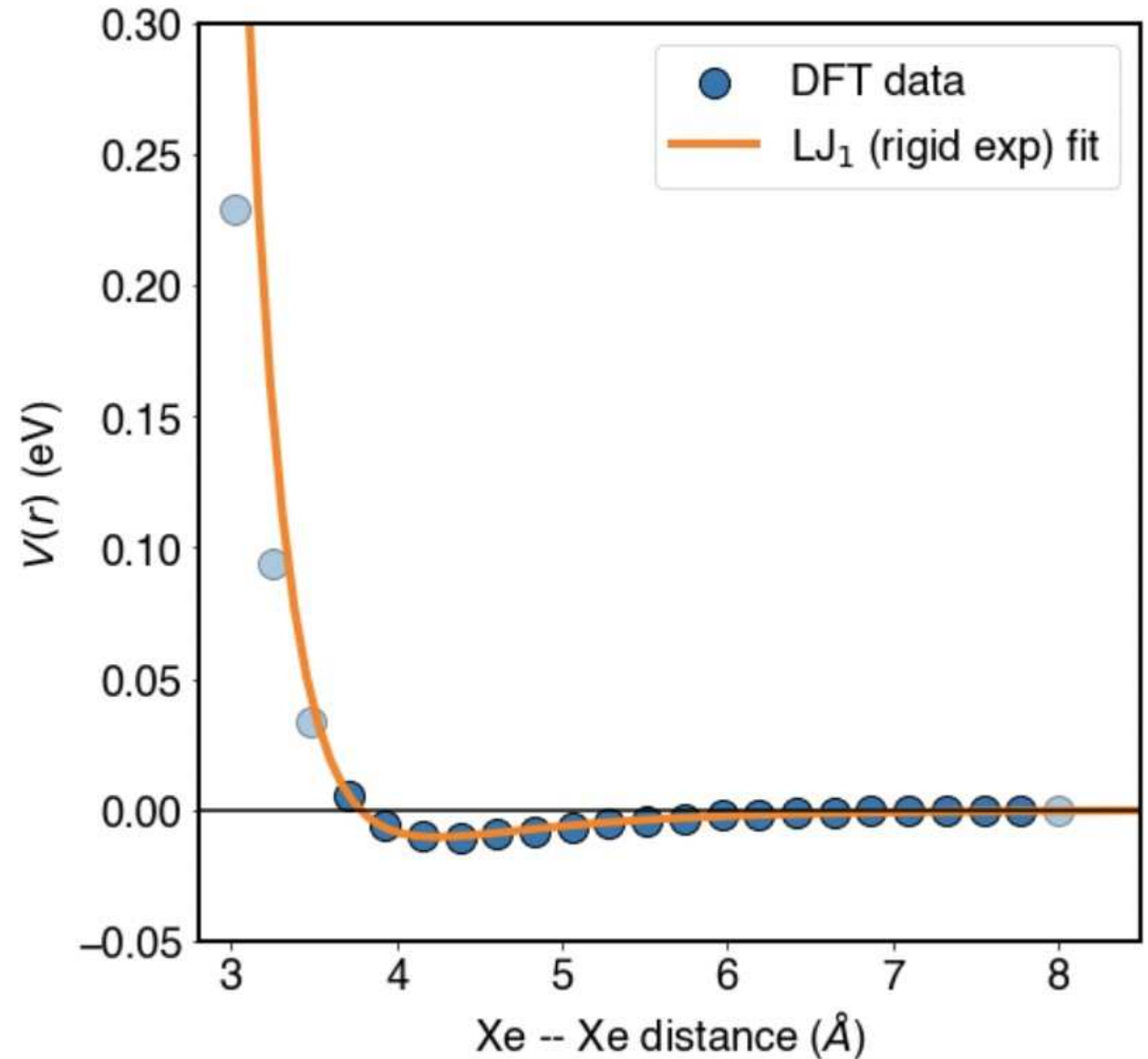
Fit results

ε (eV/atom): 0.0104

σ (Å): 3.7795

repulsive exponent: 12

binding distance: (Å): 4.242



Define fitting function

More degrees of freedom

```
def lennard_jones_2(x, epsilon, sigma, exp):  
    return 4*epsilon*((sigma/(x))**exp-(sigma/(x))**6)  
  
from scipy.optimize import curve_fit  
  
mask_fit = (LJ_data[:,0] > 3.4) & (LJ_data[:,0] < 8.0)  
popt_2, pconv = curve_fit(lennard_jones_2, LJ_data[mask_fit,0],  
                           LJ_data[mask_fit,2],  
                           p0=[0.02, 4.5, 12],  
                           bounds=[[0, 2.0, 1], [2, 10.0, 100]],  
                           maxfev=5000, ftol=1e-10)
```

Fit the data points and plot

- This is probably the best we can do with Lennard Jones

- **fcc crystal lattice parameter:**

- predicted 6 Å,

- experiment 6.2 Å

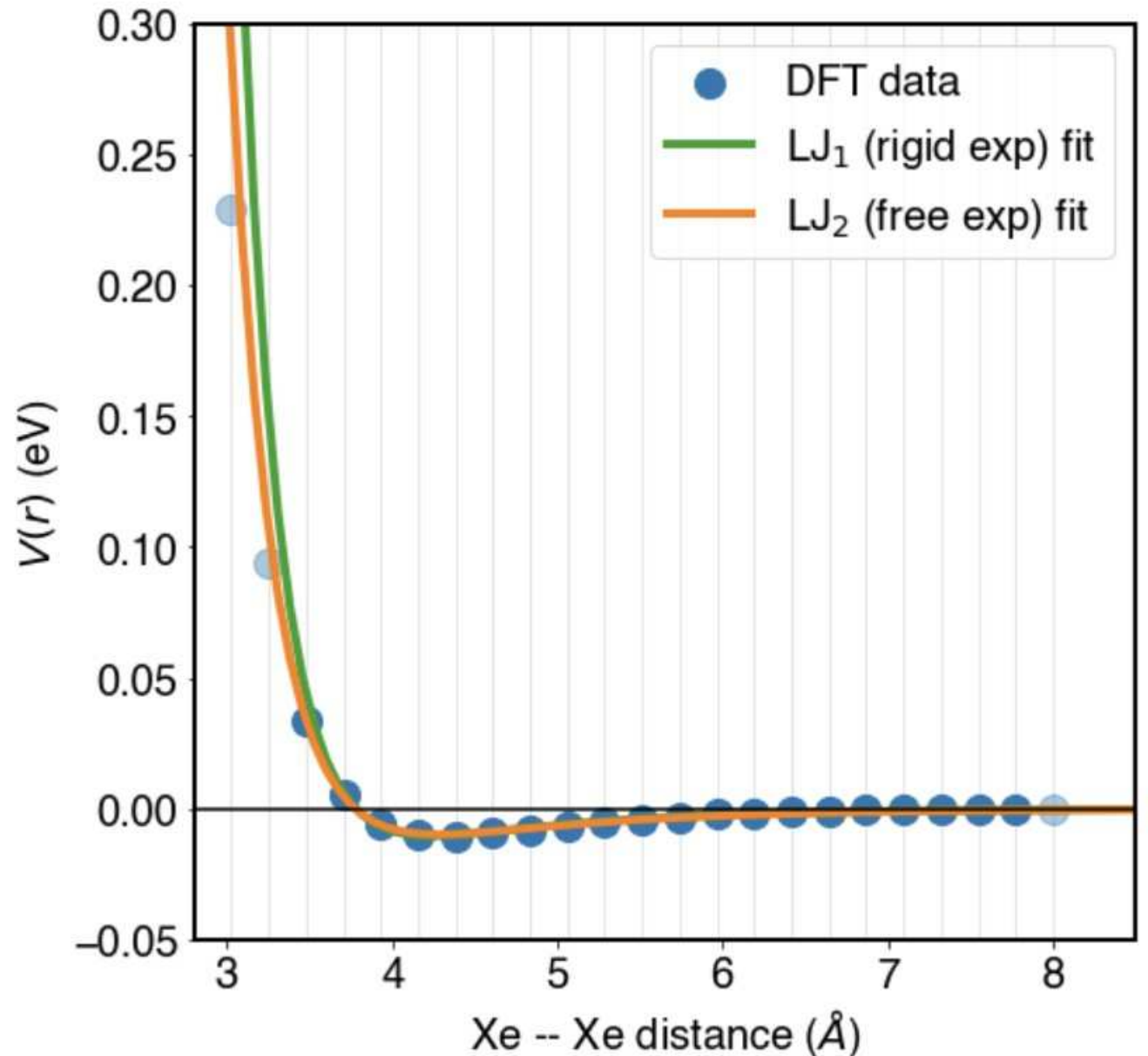
Fit results

ϵ (eV/atom): 0.0130

σ (Å): 3.7748

repulsive exponent: 10.1

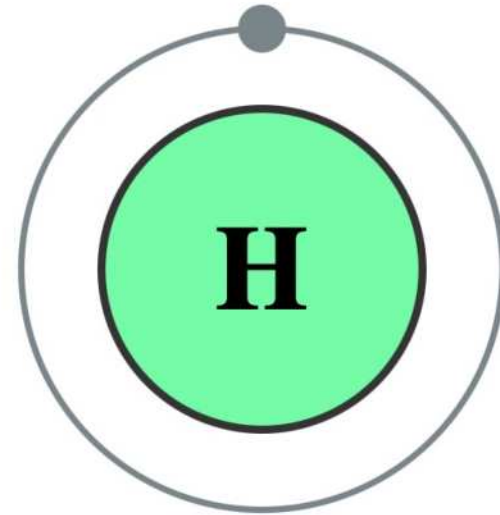
binding distance: (Å): 4.23



Case study #2: LJ Potential for H atoms

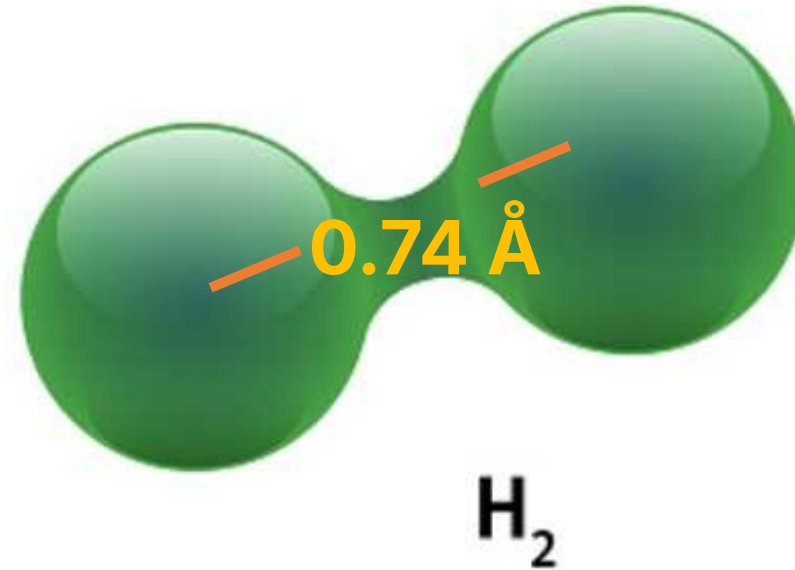
- Interatomic potential for two **hydrogen** atoms, determine ϵ and σ
- Can we do it?
- The procedure is exactly the same as before
 1. Generate a set of structures,
 2. Calculate the electronic energy with DFT
 3. Collect the data
 4. Fit

1: Hydrogen



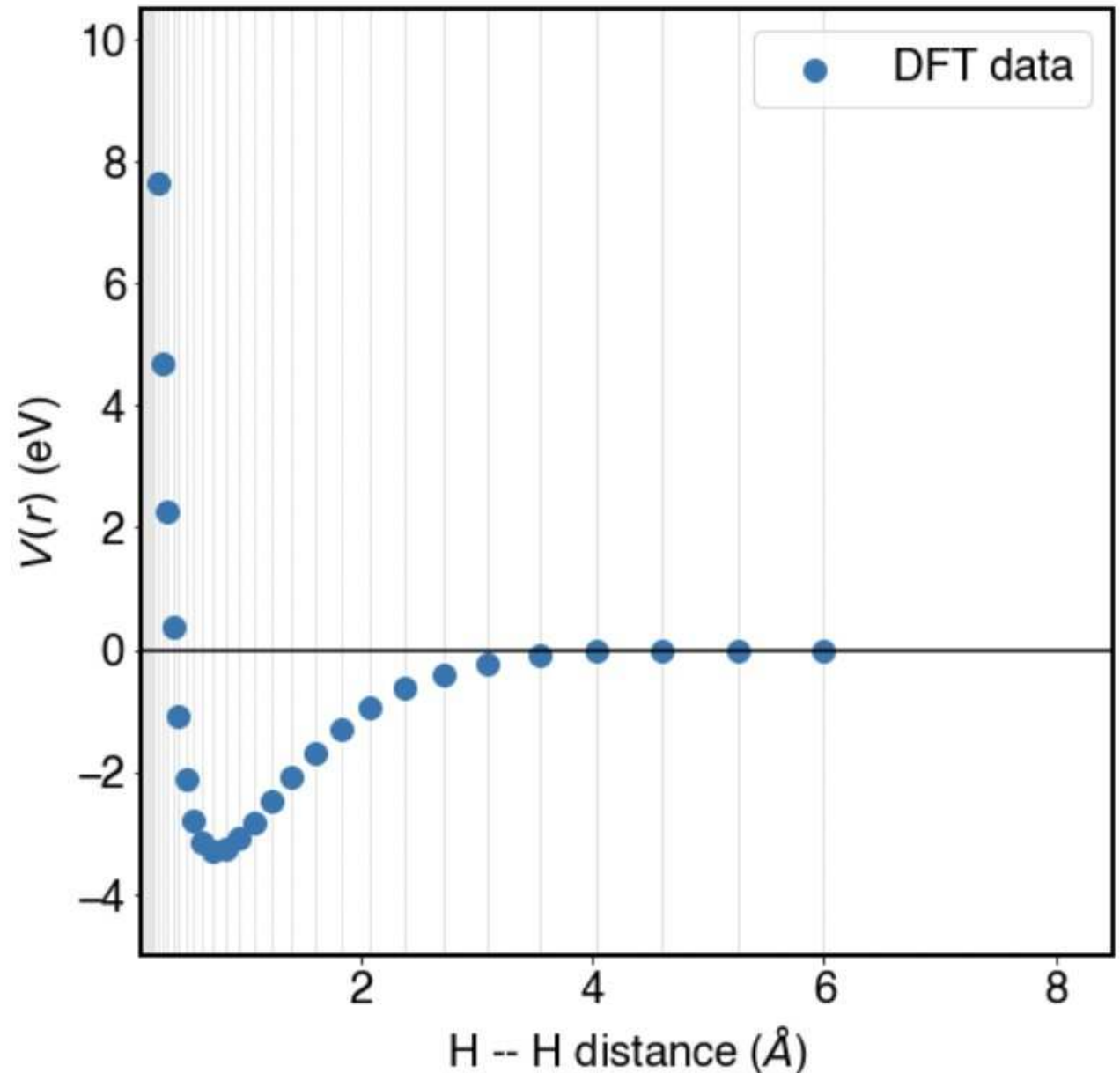
Workflow: LJ Potential for two H atoms

1. Generate a set of atomic positions for two H atoms
 - What is a sensible **range** of distances?



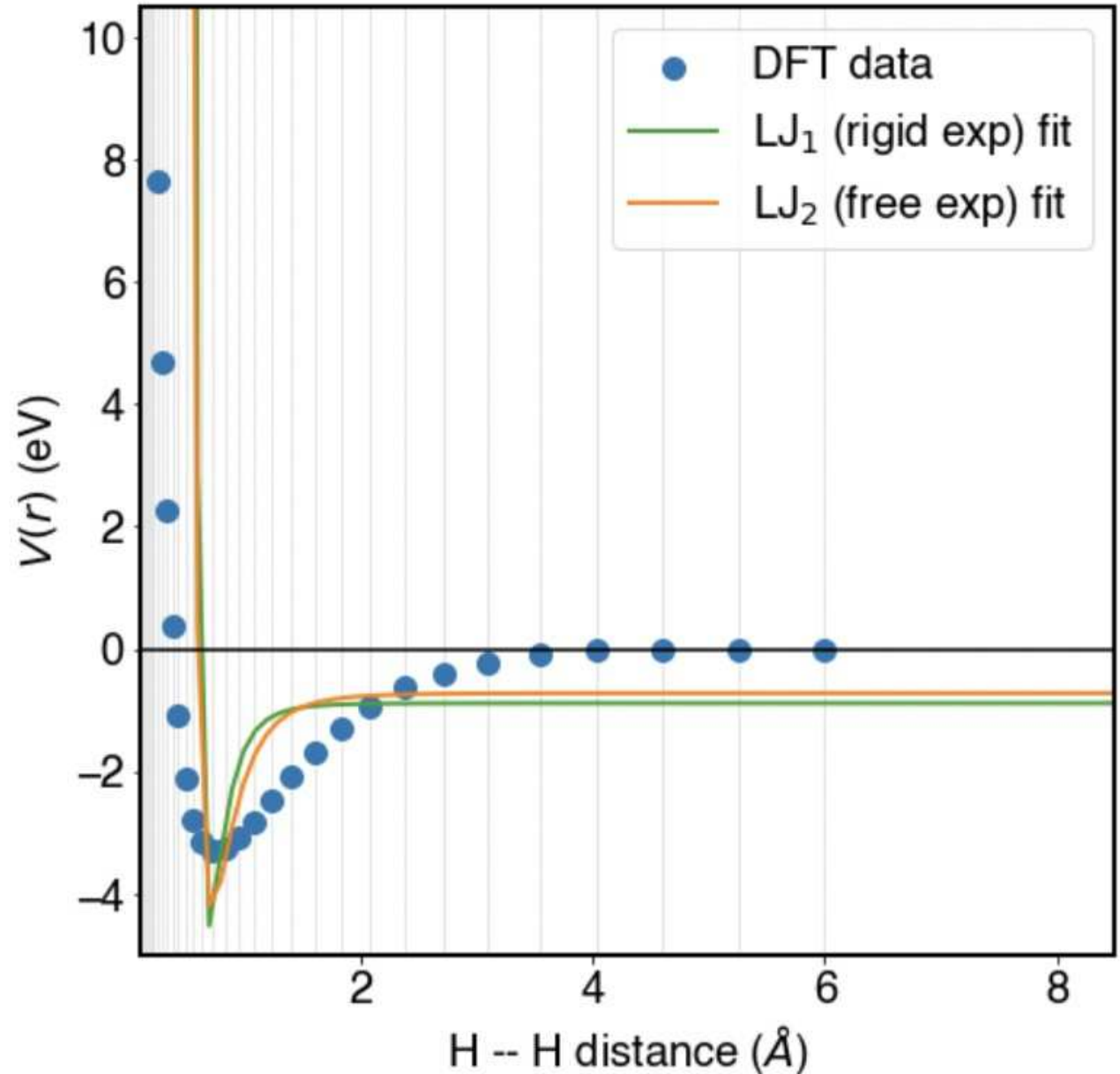
Case study #2: LJ Potential for H atoms

- How is the potential of H different from Xe?
 1. Deeper well
 2. Wider well
 3. Much shorter equilibrium distance



Case study #2: LJ Potential for H atoms

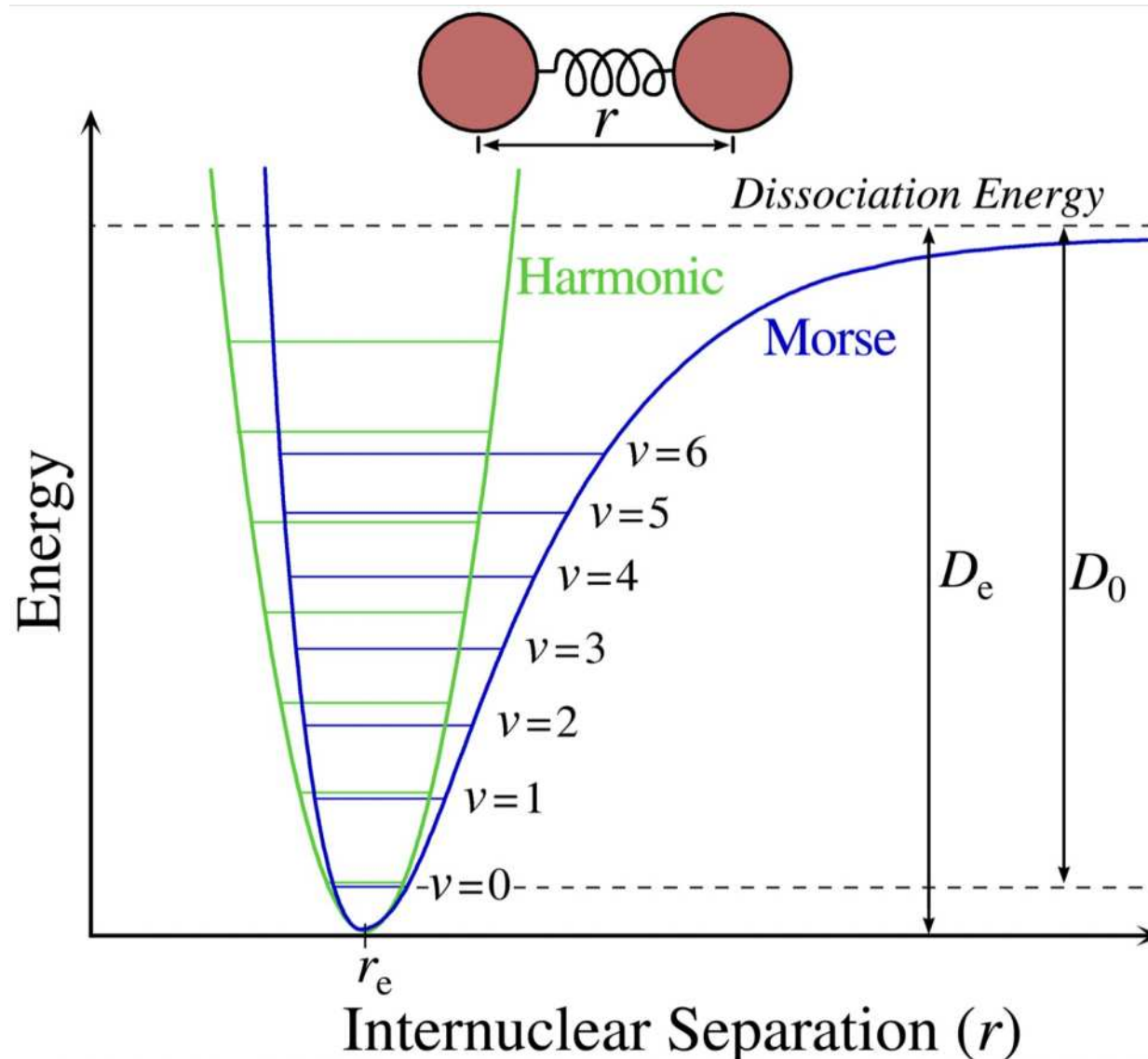
- Why is it so terrible?
 - The type of the interaction of two hydrogen atoms is **not** induced-dipole with induced dipole
 - As the two atoms get close their *electronic wavefunctions merge*, there is true Coulomb interaction, and even quantum exchange
- How to do better?



Morse Potential

- Models the *dissociation* of a diatomic molecule
- Used to describe the **anharmonic** vibrational frequencies
- Three parameters**
 - $D_e \rightarrow$ depth of the well
 - $r_e \rightarrow$ equilibrium distance
 - $a \rightarrow$ curvature at the minimum

$$V_M(r) = D_e \left(1 - e^{-a(r-r_e)}\right)^2$$



Case study #2: Morse potential for H atoms

- Define fitting function

```
def morse_1(x, d, a, r_e, shift):  
    return d*(1-np.exp(-a*(x-r_e)))**2+shift  
  
from scipy.optimize import curve_fit  
  
mask_fit = (LJ_data[:,0] > 0.3) & (LJ_data[:,0] < 8.0)  
popt_M, pconv = curve_fit(morse_1, LJ_data[mask_fit,0],  
                           LJ_data[mask_fit,2],  
                           p0=[3.5, 10, 0.74,0],  
                           bounds=[[-100, 0, 0.1,-10],  
                                    [100, np.inf, 3.0,10]],  
                           maxfev=5000, ftol=1e-10)
```

Case study #2: Morse potential for H atoms

- Good description at all energies
- Good match with experimental H – H equilibrium distance (0.74 Å)

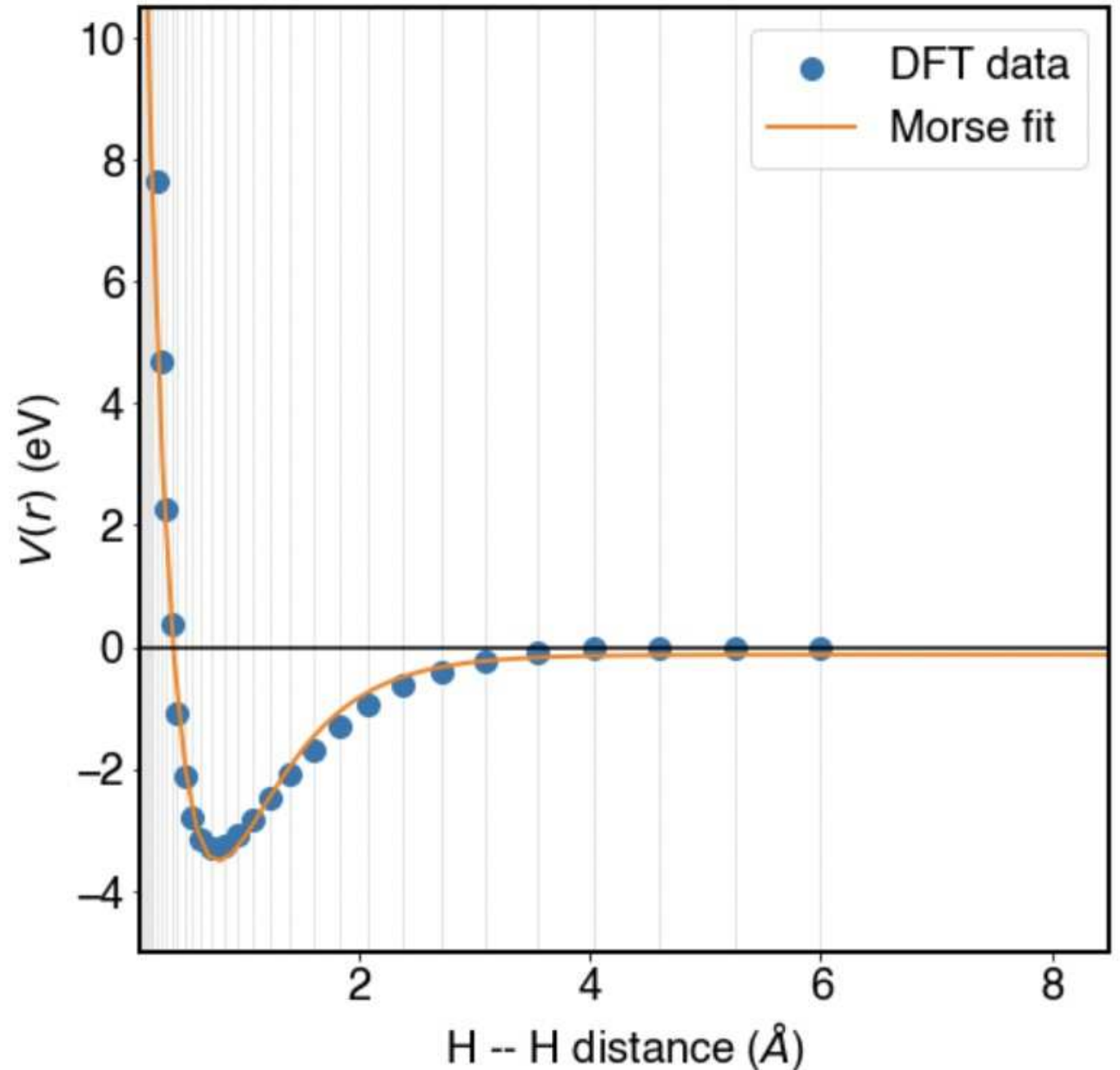
Fit results

D_e (eV/atom): 3.38

r_e (Å): 0.78

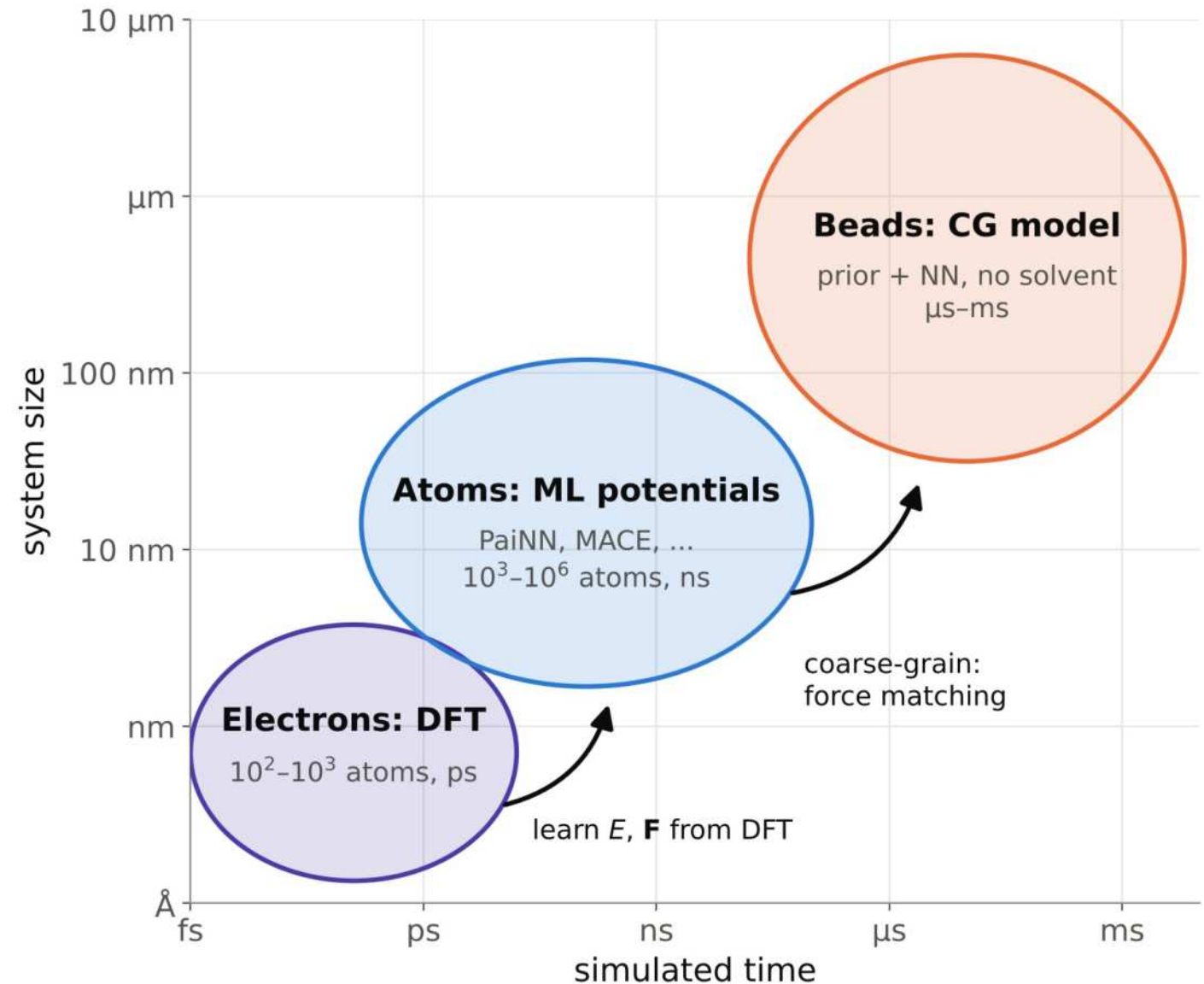
binding distance: (Å): 0.78

Shift: -3.5115



Beyond LJ and Morse: machine-learned potentials

- LJ and Morse: **a formula chosen by us**, with 2–3 parameters fitted to DFT
- Real interactions are many-body, and often we do not know the right functional form
- **Idea**: learn the energy as a function of all atomic positions directly from DFT data, with a neural network
- Then go one level up: **coarse-grain** atoms into beads, again with ML



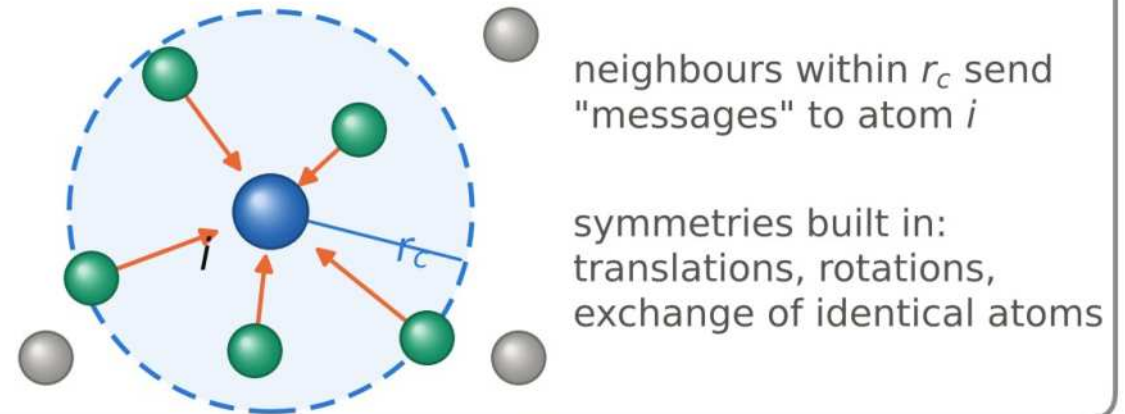
How a machine-learned potential works

- Energy = sum of atomic energies; each depends on the **neighbours within a cutoff r_c**
- Trained on DFT **energies and forces** of many configurations
- Forces $\mathbf{F} = -\nabla E$: energy is conserved in MD
- Translations, rotations and exchange of identical atoms are built in: **PaiNN** (2021), **NequIP**, **MACE** (2022)
- Near-DFT accuracy at a cost linear in N : 10^3 – 10^6 atoms for ns

1 Training data from DFT

many configurations $\{\mathbf{R}\}$: DFT energies E and forces $\mathbf{F}$

2 Each atom sees only its neighbours

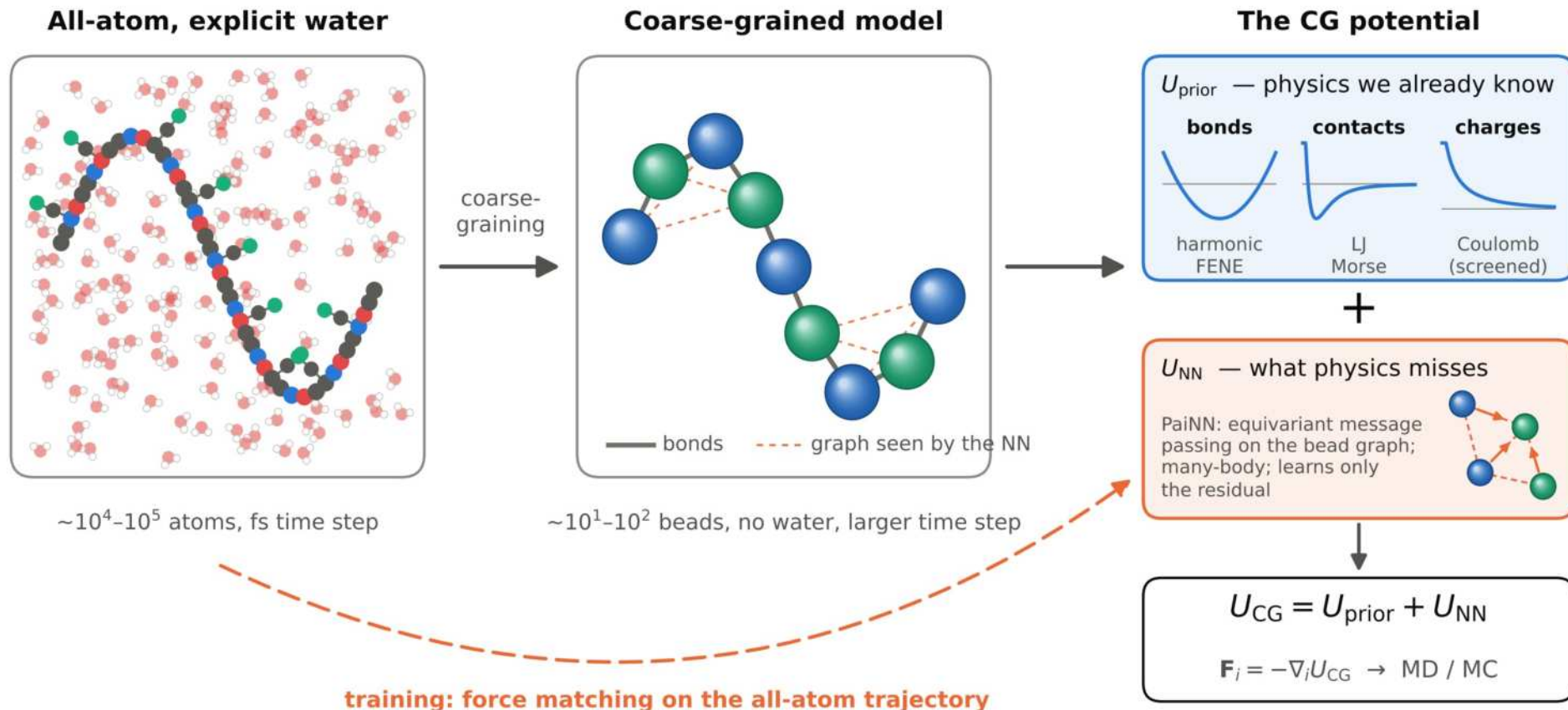


3 Neural network $\rightarrow$ energy $\rightarrow$ forces

$$E = \sum_i E_i^{\text{NN}} \quad \mathbf{F}_i = -\nabla_i E$$

Second level: coarse-graining with ML

From atoms to beads: remove the solvent, keep the physics, learn the rest



Physics prior + neural network

The potentials of today become the prior; the network learns only the correction

$$U_{\text{CG}} = U_{\text{prior}} + U_{\text{NN}}$$

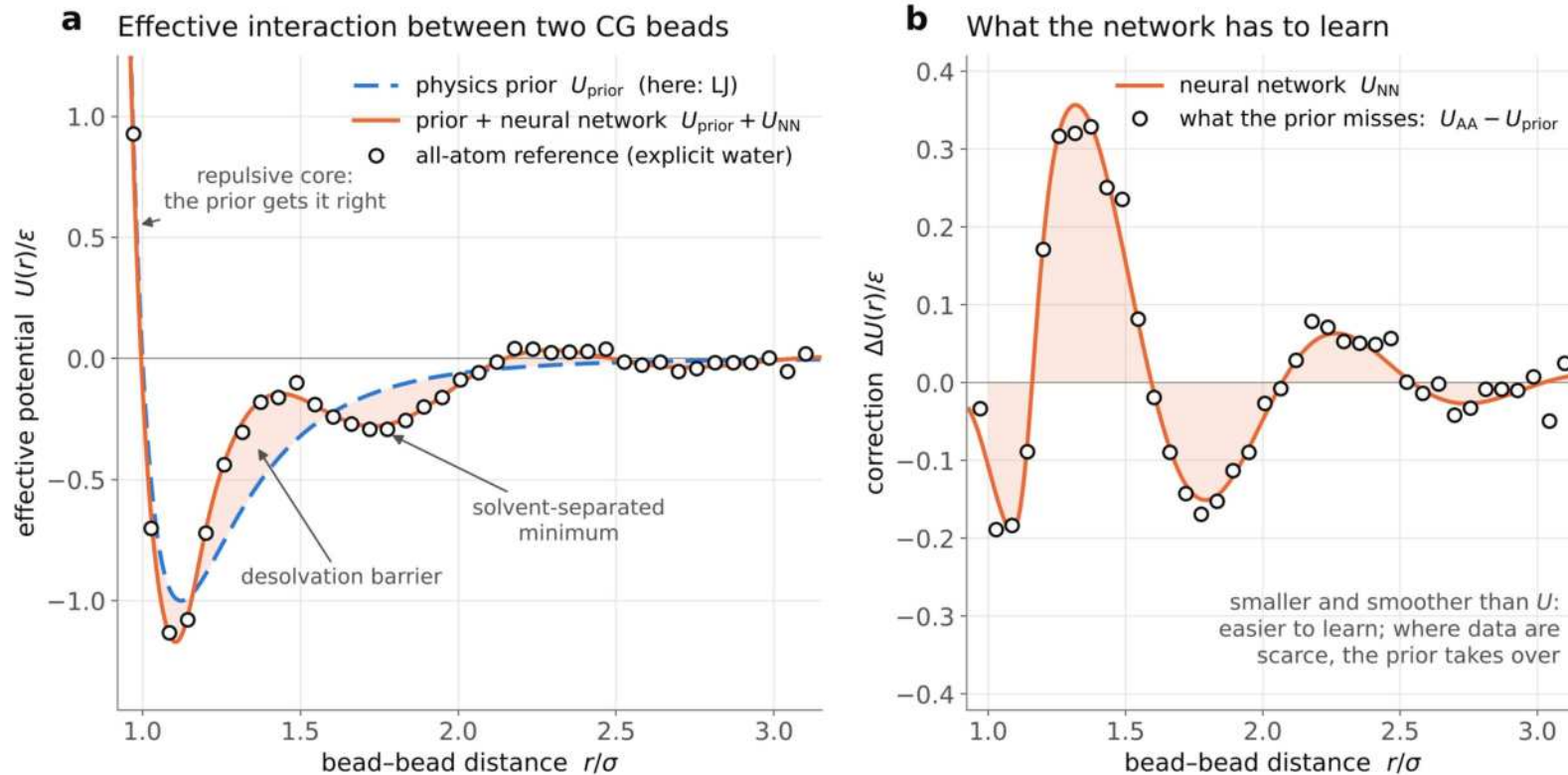


Illustration: in the real model U_{NN} is a many-body function of all bead positions (PaiNN), trained on all-atom forces.

- U_{prior} : physics we know: harmonic and FENE bonds, LJ / Morse contacts, Coulomb
- U_{NN} : what the prior misses (solvent, many-body effects)
- Trained by **force matching** on all-atom forces
- Where data are scarce, the prior keeps the model physical
- Research example: CG models of DNA G-quadruplexes with a PaiNN correction

Conclusions

1. First-principles calculations based on Density Functional Theory can be used to calculate quite accurately the parameters of a given interatomic potential
2. The choice of the potential is very important, it must capture the right physics of the interaction
 - Lennard Jones with noble gases is great, but don't use it for a binding molecule!
3. First-principles calculations correctly reproduce the interaction at the quantum level, it's up to us to model them correctly
4. We're not restricted to DFT for the quantum mechanical part (e.g. QMC, Coupled Cluster)
5. Machine-learned potentials (PaiNN, MACE, ...) learn the interaction from DFT data without fixing a functional form; a physics prior plus a neural network brings the same idea to coarse-grained models